

Datortentamen 2026-03-17

TDDE24 Funktionell och imperativ programmering del 2

Allmän information

Tentan består av totalt **6 uppgifter**. Samtliga kan ge maximalt 5 poäng.

Vad som är lätt eller svårt skiljer sig från person till person. Därför bör man inte förutsätta att de första uppgifterna är enklast, utan snabbt läsa genom alla uppgifter för att kunna prioritera arbetet. Vissa uppgifter kan också ge delpoäng för att man löser specifika delar, så om en uppgift verkar svår i sin helhet kanske du ändå vill arbeta med en del av den!

Poänggränser

Vissa studenter önskar att se i förväg hur många poäng som krävs för godkänt. Vid denna tentamen kommer 18 poäng definitivt att ge godkänt, men det är möjligt att denna gräns kommer att justeras nedåt en aning vid granskningen.

Ur universitetskanslersämbetets vägledning om rättssäker examination:

[Poänggränser anges normalt] på själva det skriftliga provet eller i andra dokument. De blir då att betrakta som riktlinjer som bör följas så långt som möjligt. UKÄ delar den bedömning som Höskoleverket och majoriteten av institutionerna uttryckte i den andra upplagan av rapporten, nämligen att justeringar kan göras när en strikt tillämpning av gränserna skulle kunna leda till en felaktig betygssättning av studenternas kunskaper och färdigheter.

Tillåtna hjälpmedel och verktyg

Följande hjälpmedel och verktyg är tillåtna / tillgängliga:

- **Penna** för att skissa lösningar på papper. (Tomma papper ska finnas tillgängliga.)
- **Python 3.11**, som vi har använt under senaste kursomgången. Detta ska finnas som kommandot `python3.11` på tentadatorerna.

I terminalfönstret *bör* det också gå att använda Python 3.11 genom att skriva `python` eller `python3`, men dessa "alias" fungerar inte nödvändigtvis om du kör program inuti en editor som `vscode`. Om du får fel version av Python inuti en editor har våra nya svarsmallar en testfunktion som ska tala om det automatiskt vid testkörningen, och det är då upp till dig själv att lösa problemet: Peka ut korrekt version av Python för editorn, eller kör helt enkelt allt med `python3.11` från kommandoraden.

- **Ett antal editorer**, inklusive `vscode` (kommandot `code` på kommandoraden) och `gvim`. Fler editorer kan finnas tillgängliga från menyer eller från kommandoradsprompten.

Editorerna har inte alla plugins installerade. Meningen är att man ska få tillgång till de grundläggande menyer och tangentbordsbindningar som man är van vid, inte att man ska ha en fullständig integrerad utvecklingsmiljö. Exempelvis kan man behöva köra program från kommandoraden, och verktyg som debuggers kanske inte fungerar.

- Standardiserade **systemprogram** i kommandoradsprompten eller menyerna.
- **Websidor** med biblioteks- och språkreferenser för Python. Enligt de generella instruktionerna för datortentor ska dessa finnas tillgängliga via en "Web Access"-ikon på skrivbordet (vi som skapar tentan kan inte själva se tentamiljön). Vid eventuella problem, räck upp handen och be tentavakterna tillkalla teknisk jour.

Otillåtna hjälpmedel; fusk och vilseledande

Otillåtna hjälpmedel inkluderar alla former av elektronisk utrustning utöver tentadatorerna, samt böcker och anteckningar.

Under datortentan arbetar du i en begränsad och övervakad datormiljö där fullständiga utvecklingsmiljöer som `PyCharm` inte är tillåtna, och där du har begränsad tillgång till nätet.

Utöver detta gäller följande:

- Man får **inte kopiera text eller lösningar direkt från andra källor**. Man måste skriva koden på egen hand och **förstå och beskriva** vad den gör.
- Man får **inte kommunicera med andra under tentan**, vare sig för att diskutera uppgifterna eller för andra syften (utom för att ställa frågor till tentavakter, så klart).
- Man får **inte göra tentasvar eller relaterad information tillgängliga för andra** på något sätt under tentans gång. Alla uppgifter ska genomföras helt individuellt.

Vi påminner om att vi är **skyldiga att anmäla** möjligt fusk eller "försök till vilseledande" till disciplinnämnden, utan att själva försöka reda ut om det faktiskt var fusk.

Svarsmallar

För varje uppgift i tentan *skickar vi med* en "svarsmall", med filnamn från ex1.py till ex6.py. **Mallen ska alltid användas som grund till din inlämnade uppgift** och du ska varken döpa om den eller skapa egna filer som lämnas in. **Kopiera** mallfilen från den skrivskyddade katalogen `given_files` till den katalog där du arbetar med uppgifterna, t.ex. `Desktop`. Skriv din kod överst i filen och dina eventuella tester inuti den fördefinierade funktionen `run_tests()` – det underlättar vid rättningen. Provkör med t.ex. `"python3.11 ex1.py"` från kommandoraden.

Mallen hjälper till med detta:

- Testar att man kör rätt version av Python, så att man inte får underliga buggar på grund av fel Pythonversion. Talar annars om vilken version som används.
- Inkluderar "tomma" funktionsdeklarationer för de funktioner du behöver skriva, så du slipper klippa och klistra och inte riskerar att skriva fel. Funktionerna består bara av kommandot `pass`, som du då ska *ersätta* med din egen kod för funktionen. Se upp så du inte får dubbla deklarerationer, både vår medskickade och en egen!

Du får givetvis skapa egna hjälpfunktioner som tillägg.

- Inkluderar eventuella assert-tester som vi redan har angivit i uppgiften, så du slipper klippa och klistra. (Du kan ändå behöva skapa egna tester, för din egen skull.)
- Ser till att testerna bara körs om man *kör* filen, inte om man *importerar* den (så som vi gör vid rättningen). Det är därför **även dina egna tester** ska vara inuti `run_tests()`.

Mallen kan se ut ungefär så här:

```
1 # Här i början lägger du din egen kod för uppgift 1.
2
3 def the_function_you_should_write(seq: list):
4     pass
5
6 def check_python_version():
7     ...färdig kod som kollar att du kör rätt version av Python...
8
9 def run_tests():
10    # De här testerna står uttryckligen som assertions på tentan.
11    print("Kör uppgiftens tester...")
12    assert the_function_you_should_write(10) == 42
13
14    # Här lägger du dina egna tester. Du kan till exempel skapa egna
15    # assertions, eller lägga till andra tester såsom enkla utskrifter
16    # av resultatet av en körning.
17    print("Kör egna tester...")
18
19    # Här kan du lägga tester där du inte vet korrekta svar men
20    # ändå kan skriva ut resultatet. Kanske det kraschar, kanske
21    # det är uppenbart fel...
22    print("Kör utskriftstester...")
23    print("Resultat 1:", the_function_you_should_write(4711))
24
25    print("Har kört alla tester")
26
27 if __name__ == '__main__':
28     check_python_version()
29     run_tests()
```

Under tentan: Lämna in uppgifter

Med **tentaklienten** (som du har fått information om i förväg och som även beskrivs i ett dokument som ska finnas tillgänglig i **given_files**) skickar du in dina svar, med en separat inlämning per uppgift. Svaren måste lämnas in innan tentans slut, då tentasystemet automatiskt stängs!

När du gör en inlämning i tentaklienten anger du också *vilken* uppgift du lämnar in (nummer 1–6). I uppdelade uppgifter lämnas del (a) och del (b) in på samma gång, *i samma fil*. Var försiktig så att du anger rätt uppgiftsnummer!

Det går bra att skicka in en lösning på samma uppgift flera gånger, och den nya inlämningen ersätter då alltid den tidigare. Utnyttja det: **Testa att lämna in någon lösning tidigt**, så du garanterat vet hur det fungerar, och **riskera inte att missa sluttid / deadline**, utan lämna in din nuvarande minst 5 minuter före sluttiden även om du tror du kan vilja utnyttja resten av tiden för att polera svaret mera.

Det finns en meddelandelogg i klienten, men såvitt vi vet kan du inte se exakt vilka filer du har bifogat. Om du är osäker på vad du har skickat in kan du skicka in rätt fil en gång till.

Precis som vid vanliga tentor kommer inga svar att granskas förrän efter tentans slut.

Under tentan: Ställa frågor

Om du har **tekniska problem** (kan inte logga in, har problem med tangentbordet, kan inte skicka in svaret på en fråga ...) kontakter du tentavakterna som vid behov kan kontakta teknisk jour.

Frågor om tentauppgifterna ställs istället via tentaklienten. Frågorna går då till examinatorn. Tentavakter och teknisk jour kan *inte* svara på tentafrågor.

Spara inte frågorna till slutet. På en vanlig tenta går man genom alla uppgifter så snart man får tillgång till dem, så man kan ställa alla eventuella frågor till examinatorn vid ett eller två korta besök i tentasalen. Under denna datortenta kan du visserligen skicka frågor när som helst, men även här kommer frågorna att besvaras "då och då" och **det kan ta någon timme att få svar** – inte minst för att det kan komma många frågor på samma gång.

Ofta kommer många frågor på slutet, så skickar du frågor för sent kan det hända att du inte hinner få svar i tid för att avsluta din lösning innan tentans slut!

Läs alltså genom uppgifterna i början och ställ frågor i god tid!

Under tentan: Informationsutskick

Information som är intressant för flera tentander kan skickas ut via tentaklienten under tentans gång. Detta brukar oftast handla om påminnelser om sådant som redan står i instruktionerna, men som några studenter har missat eller missförstått.

Utskick och svar på frågor syns bara i en meddelandelogg i tentaklienten. Det kommer inga notifikationer eller popup-meddelanden.

**Håll alltså koll på tentaklienten och dess meddelandesystem under tentan.
Klienten ger INTE automatiska notifikationer om meddelanden kommer!**

Viktiga tips om testning – missa inte poäng i onödan

Utöver rättningskriterierna (nästa sida) vill vi starkt uppmana er att tänka på detta:

- Vi ger ofta flera testfall i varje uppgift. De som är skrivna direkt i `assert`-form finns normalt också med i svarsmallen. Det kan också finnas indirekta tips som del av den löpande texten, vilket normalt *inte* läggs med i svarsmallen men kan fungera som inspiration till fler testfall. **Testa allt du kan och skapa egna variationer!** Ofta hittar man fel som faktiskt är lätta att korrigera.
- De testfall vi ger är bara exempel och täcker definitivt inte allt – man måste också utgå från beskrivningen i texten. **Skapa egna tester** som täcker fler fall!
- Det går *delvis* att testa genom att **köra implementationen med många olika indata** även om du inte vet vad korrekt svar ska bli (`print` istället för `assert`). Ibland fungerar loopar – testkör t.ex. med värden från 0 till 100! Det är inte ovanligt att koden kraschar direkt, eller skriver ut uppenbart felaktiga svar. Då har du hittat ett fel, utan att veta exakt vad det korrekta svaret skulle vara!
- Tänk på att **testa specialfall** som *tomma listor*, *tomma tupler*, *negativa tal*, med mera. Testa också *långa listor* eller *djupt nästlade listor*.
- **Läs exakt vad som står!** En *godtycklig lista* är precis vilken lista som helst, så testa med olika listor, även med sådana där elementen består av nästlade listor eller tupler eller andra datastrukturer. En *sekvens* behöver inte nödvändigtvis vara en lista, så testa även med tupler och kanske en sträng eller två. Ska det fungera för heltal $n \geq 0$ ska det också fungera för $n = 0$, så testa det. Ska funktionen *ta en sekvens och returnera en lista* får den inte returnera en tupel, även om inputsekvensen råkade vara en tupel.
- Tänk på att man ofta ska **klara godtyckliga indata**. Även om de angivna testfallen bara är heltal, kanske det står att man ska klara godtyckliga listor, och alltså vilka element som helst. **Testa då detta!** Fastna inte heller i att ett exempel bara råkar ange *positiva* tal, eller att en lista råkar vara *sorterad*. Återigen, läs vad uppgiften ska klara och skapa egna exempel som testar varierande input.

Att testa är viktigt!

Vi ser ofta *många* fel som väldigt enkelt kunde ha upptäckts och fixats om man bara *testade* sin lösning lite mer. Vad är bäst, att få 4 poäng på 4 uppgifter (16 totalt) eller att hinna med en uppgift till men få 2 poäng per uppgift på grund av bristande testning (10 totalt)?

Men testning fångar inte allt!

Grunden i uppgifterna är alltid att *läsa* och *förstå*. Testerna är inte ett facit och koden kan vara felaktig även om den klarar alla tester.

Egna tester rekommenderas men är ej obligatoriska!

Tester får lämnas in, men det är inte nödvändigt. De påverkar inte poängen.

Rättningskriterier

Brott mot följande allmänna kriterier kan resultera i poängavdrag.

- Lösningen ska givetvis vara **körbar**. Testa alltid **precis innan inlämning** så att din sista finputsning eller dina sista kommentarer inte resulterade i felaktig kod och så att koden inte kraschar när filen importeras vid granskning! Poängavdrag ges vid icke körbar kod. (Misslyckade tester och assertions är OK om de ligger i `run_tests()`, eftersom vi inte kör den funktionen vid vår betygsättning.)
- Lösningen ska följa alla de **specifika regler och villkor** som står i uppgiften.

Den ska också **fungera exakt som i körexemplen** i uppgiften, om inte texten indikerar något annat – men tänk på att koden kan vara felaktig trots att körexemplen fungerar! Lösningen ska vara **generell** och ska fungera för *alla* indata som följer uppställningen i uppgiften. Att en lösning enbart fungerar för listor med begränsad längd eller för vissa storlekar på indata är exempel på signifikanta fel.

- Funktioner och källkodsfiler ska ha **exakt samma namn** som anges i uppgiften. Vi hjälper numera till med detta genom svarsmallarna.
- Man ska kunna köra funktioner **flera gånger** med olika indata och korrekt resultat, utan att ladda om koden däremellan. Se upp med olika former av globalt tillstånd / globala variabler. Se upp med defaultargument och modifiera dem aldrig. Testa själv att köra många testfall i rad.
- Kod ska vara **lättförståelig**. Det innebär t.ex. att egna namn (på parametrar, variabler med mera) ska vara **beskrivande** och följa namnstandarden. Det innebär också att lösningen ska vara **välstrukturerad** och tillräckligt **väldokumenterad** för att en granskare **enkelt** ska förstå hur lösningen fungerar och varför den ser ut som den gör.

Vi har **inget generellt krav på docstrings**, men någon form av kommentarer kan som sagt behövas för förståelsen.

- Den implementerade lösningen ska kunna köras inom **rimliga tidsramar**. Till exempel accepteras inte en lösning som tar 1 minut för att konkatenera två strängar som är 4 bokstäver långa. Eventuella undantag anges uttryckligen i uppgiften.
- Om inte annat sägs ska funktioner **returnera** värden, inte skriva ut dem.

Avdrag för felaktig och ofullständig kod ges **även om lösningen är uppenbar** för granskaren, och även om det syns att lösningen "kan ha varit på rätt väg". Det ingår i uppgiften att se till att koden uppfyller specifikationen, och att *själv* upptäcka och åtgärda problem under tentans gång. Den inlämnade koden är ditt slutgiltiga svar, inte ett mellansteg som granskaren ska arbeta vidare med.

Lösningar som misslyckas i alltför många fall räknas normalt inte som lösningar och får 0 poäng. Det går oftast att få delpoäng för lösningar som *misslyckas* med vissa specifika fall, men inte för lösningar som *bara lyckas* med vissa specifika fall.

Tillåtet / icke-krav

Vi får ofta frågor om vad som är tillåtet i en lösning. Om **inget annat anges** i en uppgift, är följande uttryckligen **tillåtet**:

- Att lösa uppgiften **rekursivt eller iterativt, eller med en kombination** av dessa lösningsmodeller (t.ex. hybrider med rekursiva anrop och defaultargument). Med andra ord: Om inget annat sägs kräver vi inte någon specifik form av rekursion, och du kan använda en lösningsform du känner dig bekväm med.
- Att importera och använda **alla vanliga "inbyggda" funktioner** från Pythons standardbibliotek (upp till och med Python 3.11), t.ex. matematiska funktioner från `math`, högre ordningens funktioner som `map()`, och så vidare.
- Att använda **listbyggare** (list comprehensions), generatoruttryck (generator expressions), slicing (delsekvenser såsom `seq[2:5]`) och andra funktionaliteter i språket.
- Att skapa **hjälpfunktioner**, nästlade eller icke nästlade. Lösningen i sin helhet måste så klart fortfarande följa den tänkta lösningsmodellen, om en sådan är angiven.
- Att **addera defaultargument till funktioner**, så länge som funktionerna fortfarande går att anropa på sådant sätt som visas i uppgiften. Lösningen i sin helhet måste så klart fortfarande följa den tänkta lösningsmodellen, om en sådan är angiven. Defaultargument kan ibland användas på sätt som bryter mot en rekursiv lösningsmodell och är alltså problematiska om uppgiften kräver en sådan modell. Se även varningar för defaultargument under rättningskriterier.
- Att **anta att indata följer specifikationen i uppgiften**, utan några egna felkontroller. Står det t.ex. att funktionen ska ta en sekvens, behöver man inte själv kontrollera att man faktiskt får en sekvens som parameter (om inte uppgiften särskilt anger detta). Står det att funktionen ska ta en lista av heltal, får man anta att det är en lista och att den bara innehåller heltal.

Funktioner måste alltså ge *korrekta svar för korrekta indata* enligt uppgiften, men om inte annat anges får de *krascha eller ge felaktiga svar för felaktiga indata* (såsom när en funktion som bara ska hantera heltal ges en sträng som parameter).

- Att **bryta mot "ytliga" kodningsstandarder** i fråga om t.ex. mellanrum, indentering, radbrytningar, radlängd och antal blankrader, så länge som koden fortfarande är *läst och lättförståelig*. Samma gäller stil på identifierare (`snake_case`, `camelCase` med mera).
- Att **hoppa över "type hinting"** såsom `seq: list[int]` i de funktioner man själv skriver. Vi ger ofta *type hints* i funktionerna som anges i tentan, men det är bara för att hjälpa er se vilka indata som förväntas.

Detta gäller inte om uppgiften gör specifika undantag!

Uppgift 1: Invertera ett lexikon (5p)

Använd den givna filen **ex1.py** för hela denna uppgift!

Kopiera den från `given_files` till desktop eller liknande. Se ytterligare instruktioner tidigare i tentan.

Ofta har man ett lexikon (en `dict`) som mappar nycklar till värden, men behöver istället kunna slå upp vilka nycklar som hör till ett visst värde. I denna uppgift ska vi "vända på" ett lexikon.

Skriv funktionen `invert_dict(d: dict) -> dict`, som tar ett lexikon `d` och returnerar ett nytt lexikon där:

- Nycklarna utgörs av alla unika värden som fanns i `d`.
- Varje nyckel pekar på en lista av de ursprungsnycklar som hade detta värde i `d`.
- Elementen i varje lista ska ligga i samma ordning som de påträffas vid iteration över `d`.

Exempel – skapa gärna egna tester i `run_tests()`:

```
assert invert_dict({'a': 1, 'b': 1, 'c': 2}) == {1: ['a', 'b'], 2: ['c']}
assert invert_dict({'x': 'hello', 'y': 'world'}) == {'hello': ['x'], 'world': ['y']}
assert invert_dict({}) == {}
assert invert_dict({'p': 5, 'q': 5, 'r': 5}) == {5: ['p', 'q', 'r']}
```

Ytterligare villkor:

- Funktionen får **inte** modifiera sina indata.
- Du får anta att värdena i `d` är av en typ som kan användas som nycklar i ett lexikon (t.ex. heltal eller strängar, men inte listor).

Uppgift 2: Bergsekvenser (5p)

Använd den givna filen `ex2.py` för hela denna uppgift!

Kopiera den från `given_files` till desktop eller liknande. Se ytterligare instruktioner tidigare i tentan.

En sekvens av tal bildar ett "berg" om den först är *strängt växande* fram till en enda högsta topp, och därefter *strängt avtagande* ned till slutet.

Mer formellt är en lista `seq` ett berg om och endast om:

1. Listan innehåller minst 3 element.
2. Det finns exakt ett index i (där $0 < i < \text{len}(seq) - 1$) sådant att:
 - `seq[0] < seq[1] < ... < seq[i]` (strängt växande fram till toppen)
 - `seq[i] > seq[i+1] > ... > seq[-1]` (strängt avtagande efter toppen)

Skriv funktionen `is_mountain(seq: list[int]) -> bool`, som returnerar `True` om listan uppfyller ovanstående villkor och annars `False`.

Exempel:

```
assert is_mountain([1, 3, 5, 4, 2]) == True
assert is_mountain([0, 10, 5, 2]) == True
assert is_mountain([1, 2, 2, 3, 1]) == False    # Ej strängt växande (2 följt av 2)
assert is_mountain([1, 2, 3]) == False          # Saknar avtagande del
assert is_mountain([3, 2, 1]) == False          # Saknar växande del
assert is_mountain([1, 3, 2, 4, 1]) == False    # Mer än en topp
assert is_mountain([]) == False
assert is_mountain([5]) == False
```

Ytterligare villkor:

- Funktionen får **inte** modifiera sina indata.

Uppgift 3: Sökvägar i nästlade listor (5p)

Använd den givna filen `ex3.py` för hela denna uppgift!

Kopiera den från `given_files` till desktop eller liknande. Se ytterligare instruktioner tidigare i tentan.

En nästlad lista kan ses som ett träd, där varje grundelement (som inte i sig är en lista) kan beskrivas med en sekvens av index som anger hur man tar sig ner genom strukturen.

Till exempel, i listan `[10, [20, 30]]`:

- Elementet `10` nås genom index `0` i den yttersta listan.
- Elementet `20` nås genom index `1` (den inre listan) följt av `0` – `listan[1][0]`.
- Elementet `30` nås genom index `1` följt av `1`.

Vi representerar dessa "sökvägar" som tupler av heltal: `(0,)`, `(1, 0)` och `(1, 1)`. Notera att `(0,)` är Pythons syntax för en tupel med ett enda element.

Skriv funktionen `enumerate_nested(nest: list) -> dict`, som tar en nästlad lista av godtyckligt djup och returnerar ett lexikon (`dict`). I detta lexikon ska:

- Varje nyckel vara en sökväg (tupel av index) till ett grundelement.
- Varje värde vara just det grundelement som sökvägen pekar ut.

Exempel:

```
assert enumerate_nested([10, [20, 30]]) == {(0,): 10, (1, 0): 20, (1, 1): 30}
assert enumerate_nested([["a"], "b"]) == {(0, 0): 'a', (1,): 'b'}
assert enumerate_nested([], 42) == {(1,): 42}
assert enumerate_nested([]) == {}
```

Tips och ledtrådar:

- Listor kan nästlas i godtyckligt många nivåer, vilket gör detta till ett problem som lämpar sig för *rekursion*.
- Det kan vara klokt att skapa en inre hjälpfunktion som har en extra parameter för den "aktuella sökvägen" och som bygger vidare på den vid varje rekursivt anrop.
- Bara listor räknas som nästlade strukturer. Andra element (t.ex. strängar, tupler, heltal) är grundelement och ska *inte* utforskas vidare på djupet.

Ytterligare villkor:

- Funktionen får **inte** modifiera sina indata.

Uppgift 4: Högre ordningens validering (5p)

Använd den givna filen `ex4.py` för hela denna uppgift!

Kopiera den från `given_files` till desktop eller liknande. Se ytterligare instruktioner tidigare i tentan.

Ibland vill man kontrollera om data uppfyller flera regler samtidigt. I stället för en stor, komplicerad `if`-sats kan man bygga en "regelmotor" med hjälp av högre ordningens funktioner.

Deluppgift 4a (3p):

Skriv funktionen `make_validator(rules: list)`.

- Parametern `rules` är en lista av funktioner (*predikat*). Varje funktion tar exakt ett argument och returnerar `True` eller `False`.
- `make_validator` ska returnera en *ny funktion* (en "validerare").
- Den returnerade valideraren tar ett värde `x` och returnerar `True` om *alla* funktioner i `rules` returnerar `True` för `x`. Om någon returnerar `False` ska valideraren returnera `False`.
- Om `rules` är en tom lista ska valideraren alltid returnera `True`.

Exempel för 4a:

```
is_even = lambda x: x % 2 == 0
```

```
is_positive = lambda x: x > 0
```

```
valid_num = make_validator([is_even, is_positive])
```

```
assert valid_num(4) == True
```

```
assert valid_num(3) == False    # Inte jämnt
```

```
assert valid_num(-2) == False   # Inte positivt
```

```
always_ok = make_validator([])
```

```
assert always_ok("anything") == True
```

Deluppgift 4b (2p):

Skriv funktionen `filter_valid(seq: list, validator_func) -> list`, som tar en lista `seq` och en validerarfunktion (som den i 4a) och returnerar en ny lista med de element som passerar valideringen.

För full poäng ska detta implementeras med *exakt en rad* inuti funktionskroppen, med hjälp av en listbyggare (*list comprehension*).

Exempel för 4b (med `valid_num` definierad som ovan):

```
assert filter_valid([-2, 2, 3, 4, 5], valid_num) == [2, 4]
```

```
assert filter_valid([], valid_num) == []
```

Uppgift 5: Robotens väg (5p)

Använd den givna filen `ex5.py` för hela denna uppgift!

Kopiera den från `given_files` till desktop eller liknande. Se ytterligare instruktioner tidigare i tentan.

Tänk dig en robot som startar på koordinaten $(0, 0)$ i ett oändligt tvådimensionellt rutnät. Roboten får en sekvens av rörelseinstruktioner i form av en sträng, där varje tecken motsvarar ett steg:

- `'N'` (norr): y -koordinaten ökar med 1
- `'S'` (söder): y -koordinaten minskar med 1
- `'E'` (öster): x -koordinaten ökar med 1
- `'W'` (väster): x -koordinaten minskar med 1

Skriv funktionen `crosses_own_path(moves: str) -> bool`, som returnerar `True` om roboten vid något tillfälle besöker en koordinat som den *redan har varit på* (inklusive startkoordinaten $(0, 0)$). Om roboten aldrig besöker samma koordinat två gånger ska funktionen returnera `False`.

Exempel:

```
assert crosses_own_path("NESW") == True
# Startar (0,0) -> (0,1) -> (1,1) -> (1,0) -> (0,0). Tillbaka till start!
```

```
assert crosses_own_path("NNN") == False
# (0,0) -> (0,1) -> (0,2) -> (0,3). Korsar aldrig sin väg.
```

```
assert crosses_own_path("NESEN") == False
```

```
assert crosses_own_path("EW") == True
# (0,0) -> (1,0) -> (0,0). Tillbaka till start.
```

```
assert crosses_own_path("") == False
# Roboten rör sig inte -- har alltså inte korsat sin egen väg.
```

Tips:

- Du behöver hålla reda på vilka koordinater roboten har besökt hittills. Datatypen *mängd* (`set`) är mycket effektiv för att lagra och slå upp besökta koordinater: En koordinat kan representeras som en tupel `(x, y)`.
- Glöm inte att startkoordinaten $(0, 0)$ räknas som besökt redan innan roboten tar sitt första steg!

Uppgift 6: Datatyp för mängder (5p)

Använd den givna filen `ex6.py` för hela denna uppgift!

Kopiera den från `given_files` till desktop eller liknande. Se ytterligare instruktioner tidigare i tentan.

I denna uppgift ska du definiera en egen datatyp som fungerar ungefär som Pythons inbyggda `set` (mängd), men som internt lagrar elementen i en vanlig Python-lista. Vi kallar denna datatyp `ListSet`.

Det är uttryckligen förbjudet att använda Pythons inbyggda `set` i denna uppgift!

I `ex6.py` finns följande startpunkt (ändra den inte):

```
from typing import NamedTuple, Any
ListSet = NamedTuple("ListSet", [("elements", list[Any])])
```

En `ListSet` har ett fält `elements` (en vanlig lista). En mängd får *aldrig* innehålla dubletter, så du måste säkerställa att `elements` aldrig innehåller samma värde mer än en gång.

Funktioner att implementera:

1. `new_listset()` och `listset_contains(ls, val)` (1p sammanlagt)

`new_listset()` skapar och returnerar en ny tom `ListSet`.

`listset_contains(ls: ListSet, val: Any)` returnerar `True` om `val` finns i mängden, annars `False`.

2. `listset_add(ls: ListSet, val: Any)` (2p)

Lägger till `val` i mängden `ls`. Om värdet redan finns ska ingenting hända. Funktionen modifierar listan *på plats* och ska *inte* returnera något.

3. `listset_union(ls1: ListSet, ls2: ListSet)` (2p)

Returnerar en helt *ny* `ListSet` som innehåller alla element som finns i antingen `ls1`, `ls2` eller båda. Varken `ls1` eller `ls2` får modifieras.

Partiella exempel:

```
ls1 = new_listset()
assert isinstance(ls1, ListSet)

listset_add(ls1, 10)
listset_add(ls1, 20)
listset_add(ls1, 10) # Dubblett, ska ignoreras

assert listset_contains(ls1, 10) == True
assert listset_contains(ls1, 99) == False
assert len(ls1.elements) == 2
```

```
ls2 = new_listset()
listset_add(ls2, 20)
listset_add(ls2, 30)

ls_union = listset_union(ls1, ls2)
assert listset_contains(ls_union, 10)
assert listset_contains(ls_union, 20)
assert listset_contains(ls_union, 30)
assert len(ls_union.elements) == 3
```